

SIEM SecureBank SA

Elastic Stack + Wazuh + Kibana Security

Guide de dépannage complet — Problèmes réels et solutions

■ SecureBank SA

■ Elastic 8.19.14

■ Wazuh 4.14.4

■ Ubuntu 20.04

■ ARCHITECTURE DU LAB

VM	Rôle	IP LAN (ens38)	IP WAN (ens33)	OS
VM-SIEM	Elasticsearch + Kibana + Wazuh Manager + Filebeat	192.168.1.200	10.12.54.79	Ubuntu 20.04
VM-WEB	Cible (DVWA) + Wazuh Agent	192.168.1.199	10.12.54.80	Ubuntu 20.04
pfSense	Firewall + Suricata IDS	192.168.1.1 (LAN)	10.12.54.84 (WAN)	FreeBSD
Kali Linux	Attaquant Red Team	192.168.1.130	-	Kali 2024

01 — PROBLÈME : kibana.yml CORROMPU

Symptôme

Kibana crashe au démarrage avec `YAMLEException`. Cause : utilisation répétée de `tee -a` qui accumule des fragments mal formatés dans le fichier de configuration.

■ Erreur typique dans les logs :

```
FATAL CLI ERROR YamleException: bad indentation of a mapping entry Duplicate field
'xpack.security.enabled'
```

■ Solution — Réécrire kibana.yml proprement

```
# Ne JAMAIS utiliser tee -a – toujours réécrire en entier sudo tee
/etc/kibana/kibana.yml > /dev/null <<'EOF' server.host: "0.0.0.0" server.port: 5601
elasticsearch.hosts: ["http://127.0.0.1:9200"] elasticsearch.username: "kibana_system"
elasticsearch.password: "KibanaSystem@2026" xpack.encryptedSavedObjects.encryptionKey:
"ccaa718dfd4321546397a4986156c3122cdf385688782571730d038161e8e7eb" telemetry.enabled:
false telemetry.optIn: false EOF sudo systemctl restart kibana sleep 45
```

■ Règle importante

■ Dans Kibana 8.x, la clé **xpack.security.enabled** n'existe PAS dans kibana.yml — c'est Elasticsearch qui contrôle la sécurité. L'ajouter fait crasher Kibana avec : "definition for this key is missing".

02 — PROBLÈME : elasticsearch.yml CORROMPU

Symptôme

Elasticsearch crashe avec exit code 78 (CONFIG) — champ dupliqué dans le YAML.

■ Erreur typique :

```
Caused by: com.fasterxml.jackson.core.JsonParseException: Duplicate field
'xpack.security.enabled'
```

■ Solution — Réécrire elasticsearch.yml

```
sudo tee /etc/elasticsearch/elasticsearch.yml > /dev/null <<'EOF' cluster.name:
securebank-soc node.name: node-1 network.host: 0.0.0.0 http.port: 9200 path.data:
/var/lib/elasticsearch path.logs: /var/log/elasticsearch discovery.type: single-node
xpack.security.enabled: true xpack.security.http.ssl.enabled: false
xpack.security.transport.ssl.enabled: false EOF sudo systemctl restart elasticsearch
```

■ Données existantes incompatibles avec security

Si ES a des données créées SANS security, activer security cause un crash. Solution : vider les données.

```
sudo systemctl stop elasticsearch sudo mv /var/lib/elasticsearch
/var/lib/elasticsearch.bak sudo mkdir -p /var/lib/elasticsearch sudo chown -R
elasticsearch:elasticsearch /var/lib/elasticsearch sudo systemctl start elasticsearch
```

03 — ACTIVER LA SÉCURITÉ ELASTICSEARCH

Pourquoi activer la sécurité ?

Kibana Security Solution (alertes, règles SIEM) **nécessite obligatoirement** `xpack.security.enabled: true` côté Elasticsearch. Sans cela, les alertes affichent "Detection engine permissions required" et "403 Forbidden".

Étape 1 — Créer les fichiers realm manquants

```
# Ces fichiers sont requis pour le file realm d'Elasticsearch sudo touch
/etc/elasticsearch/users sudo touch /etc/elasticsearch/users_roles sudo chown
root:elasticsearch /etc/elasticsearch/users /etc/elasticsearch/users_roles sudo chmod
660 /etc/elasticsearch/users /etc/elasticsearch/users_roles
```

Étape 2 — Initialiser le mot de passe elastic

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic # Noter le mot
de passe généré automatiquement # Exemple : New value: 0jEPH3_qndJzGwLvE9QB
```

Étape 3 — Initialiser kibana_system

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u kibana_system #
Exemple : New value: qUIlkbB+0CHt58buiuOt
```

Étape 4 — Tester la connexion

```
curl -s http://localhost:9200/ -u elastic:MOT_DE_PASSE_GENERE | python3 -m json.tool |
grep cluster_name # Résultat attendu : "cluster_name": "securebank-soc"
```

Étape 5 — Mettre à jour kibana.yml

```
sudo tee /etc/kibana/kibana.yml > /dev/null <<'EOF' server.host: "0.0.0.0" server.port:
5601 elasticsearch.hosts: ["http://127.0.0.1:9200"] elasticsearch.username:
"kibana_system" elasticsearch.password: "VOTRE_MOT_DE_PASSE_KIBANA_SYSTEM"
xpack.encryptedSavedObjects.encryptionKey:
"ccaa718dfd4321546397a4986156c3122cdf385688782571730d038161e8e7eb" telemetry.enabled:
false telemetry.optIn: false EOF sudo systemctl restart kibana sleep 45
```

Connexion à Kibana

URL	http://localhost:5601 ou http://192.168.1.200:5601
Username	elastic
Password	Le mot de passe généré à l'étape 2

04 — PROBLÈME : RÉSEAU VM DISPARU

Symptôme

Après un redémarrage ou une manipulation, les interfaces réseau (ens33, ens38) sont en état DOWN et les IPs ont disparu. NetworkManager écrase la configuration netplan.

Diagnostic

```
ip a # Si ens33 et ens38 sont DOWN sans adresse IP → problème réseau # Vérifier le
fichier netplan cat /etc/netplan/01-netcfg.yaml
```

Solution — Configurer netplan de façon permanente

```
# Désactiver NetworkManager (cause du problème) systemctl stop NetworkManager systemctl
disable NetworkManager # Configurer netplan avec renderer: networkd sudo tee
/etc/netplan/01-netcfg.yaml > /dev/null <<'EOF' network: version: 2 renderer: networkd
ethernets: ens33: dhcp4: false addresses: [10.12.54.79/16] gateway4: 10.12.0.1
nameservers: addresses: [8.8.8.8] ens38: dhcp4: false addresses: [192.168.1.200/24] EOF
sudo netplan apply ip a | grep 'inet '
```

Restauration rapide si IPs disparues

```
# Remonter les interfaces manuellement ip link set ens33 up ip link set ens38 up ip addr
add 10.12.54.79/16 dev ens33 2>/dev/null ip addr add 192.168.1.200/24 dev ens38
2>/dev/null # Vérifier ip a | grep 'inet '
```

05 — PROBLÈME : WAZUH AGENT NE DÉTECTE PAS

Symptôme

Les attaques SSH depuis Kali arrivent sur VM-WEB (visibles dans auth.log) mais aucune alerte n'apparaît dans Kibana Security.

Chaîne de détection complète

Étape	Composant	Vérification
1	Kali attaque VM-WEB	tail -f /var/log/auth.log grep "192.168.1.130"
2	Wazuh agent lit auth.log	grep auth.log /var/ossec/etc/ossec.conf
3	Agent envoie vers Manager	nc -zv 192.168.1.200 1514
4	Manager génère alerte	tail -f /var/ossec/logs/alerts/alerts.json
5	Filebeat → Elasticsearch	curl localhost:9200/wazuh-alerts-*/_count
6	Règle Kibana détecte	Security → Alerts → Refresh

Fix 1 — Ajouter auth.log à Wazuh (sur VM-WEB)

```
# auth.log manque dans la config Wazuh par défaut ! sudo sed -i 's|</ossec_config>|
<localfile>\n <log_format>syslog</log_format>\n <location>/var/log/auth.log</location>\n
</localfile>\n</ossec_config>|' /var/ossec/etc/ossec.conf # Vérifier grep -A3 'auth.log'
/var/ossec/etc/ossec.conf # Redémarrer sudo systemctl restart wazuh-agent
```

Fix 2 — Ajouter Apache logs (pour détecter Nikto)

```
sudo sed -i 's|</ossec_config>| <localfile>\n <log_format>apache</log_format>\n
<location>/var/log/apache2/access.log</location>\n </localfile>\n</ossec_config>|'
```

```
/var/ossec/etc/ossec.conf sudo systemctl restart wazuh-agent
```

■ Fix 3 — Vérifier la connexion agent → manager

```
# Sur VM-WEB nc -zv 192.168.1.200 1514 # doit dire "succeeded" nc -zv 192.168.1.200 1515  
# doit dire "succeeded" # Sur VM-SIEM /var/ossec/bin/agent_control -l # web01-securebank  
doit être "Active"
```

06 — CONFIGURATION FILEBEAT (VM-SIEM)

Configuration complète filebeat.yml

```
sudo tee /etc/filebeat/filebeat.yml > /dev/null <<'EOF' filebeat.inputs: - type: log
enabled: true paths: - /var/ossec/logs/alerts/alerts.json json.keys_under_root: true
json.add_error_key: true fields: module: wazuh fields_under_root: true - type: syslog
enabled: true protocol.udp: host: "0.0.0.0:514" fields: module: pfsense
fields_under_root: true output.elasticsearch: hosts: ["http://localhost:9200"] username:
"elastic" password: "VOTRE_MOT_DE_PASSE" index: "wazuh-alerts-%{+yyyy.MM.dd}"
setup.ilm.enabled: false setup.template.name: "wazuh-alerts" setup.template.pattern:
"wazuh-alerts-*" setup.kibana: host: "localhost:5601" username: "elastic" password:
"VOTRE_MOT_DE_PASSE" EOF sudo systemctl restart filebeat systemctl status filebeat |
grep Active
```

■ Champ IP source dans Wazuh

Wazuh n'utilise PAS le champ ECS standard source.ip — il utilise **data.srcip**. Dans Kibana Alerts, ajouter la colonne **data.srcip** pour voir l'IP de l'attaquant.

07 — RÈGLES DE DÉTECTION KIBANA

Activer toutes les règles prédéfinies Elastic

```
# Charger les règles prédéfinies (1000+ règles) curl -s -X PUT \
"http://localhost:5601/api/detection_engine/rules/prepackaged" \ -u
elastic:VOTRE_MOT_DE_PASSE \ -H "kbn-xsrf: true" | python3 -m json.tool | grep -E
'rules_installed|rules_updated'
```

Créer une règle personnalisée SecureBank via API

```
curl -s -X POST "http://localhost:5601/api/detection_engine/rules" \ -u
elastic:VOTRE_MOT_DE_PASSE \ -H "kbn-xsrf: true" \ -H "Content-Type: application/json"
-d '{ "type": "query", "language": "kuery", "index": ["wazuh-alerts-*"], "query":
"data.srcip: *", "name": "SecureBank — Toutes Attaques", "description": "Detecte toutes
attaques reseau", "severity": "critical", "risk_score": 95, "enabled": true, "interval":
"1m", "from": "now-6h" }'
```

Règle EQL — Kill Chain SecureBank (TP)

```
# Dans Kibana : Security → Rules → Create → Event Correlation sequence by source.ip
with maxspan=30m /* Phase 1 : Brute-force SSH */ [authentication where event.outcome ==
"failure" and destination.port == 22] [authentication where event.outcome == "failure"
and destination.port == 22] /* Phase 2 : SQLi */ [network where event.module ==
"suricata" and suricata.eve.alert.category like "*sql*"]
```

08 — VÉRIFICATIONS RAPIDES — CHECKLIST

Elasticsearch opérationnel

```
curl -s http://localhost:9200/_cluster/health -u elastic:PASS | python3 -m json.tool |  
grep status # Attendu : "status": "green" ou "yellow"
```

Kibana opérationnel

```
curl -s http://localhost:5601/api/status | python3 -m json.tool | grep '"level"' | head  
-1 # Attendu : "level": "available"
```

Wazuh Manager actif

```
systemctl status wazuh-manager | grep Active # Attendu : Active: active (running)
```

Wazuh Agent connecté

```
/var/ossec/bin/agent_control -l # Attendu : web01-securebank ... Active
```

Filebeat actif

```
systemctl status filebeat | grep Active # Attendu : Active: active (running)
```

Index Wazuh dans ES

```
curl -s "http://localhost:9200/_cat/indices?v" -u elastic:PASS | grep wazuh # Attendu :  
wazuh-alerts-YYYY.MM.DD
```

Alertes Kali dans ES

```
curl -s "http://localhost:9200/wazuh-alerts-*/_count" -u elastic:PASS \ -H  
"Content-Type: application/json" \ -d  
'{"query":{"match":{"data.srcip":"192.168.1.130"}}}' # Attendu : count > 0
```

Port 1514 Wazuh ouvert

```
ss -ltnp | grep 1514 # Attendu : LISTEN sur 0.0.0.0:1514
```

09 — SIMULATION ATTAQUE SANS VM-KALI

Si Kali n'est pas disponible ou si le trafic ne traverse pas pfSense, simuler les alertes directement dans Elasticsearch via API.

Script de simulation complet (3 phases)

```
cat > /tmp/simulate_attack.sh << 'SCRIPT' #!/bin/bash ES="http://localhost:9200"  
AUTH="elastic:VOTRE_MOT_DE_PASSE" IDX="wazuh-alerts-$(date +%Y.%m.%d)"  
ATTACKER="185.220.101.47" TARGET="192.168.1.199" # Phase 1 – Nmap scan curl -s -X POST  
"$ES/$IDX/_doc" -u "$AUTH" \ -H "Content-Type: application/json" -d "{ \"@timestamp\":  
\"$(date -u +%Y-%m-%dT%H:%M:%SZ)\", \"event.module\": \"suricata\", \"source.ip\":  
\"$ATTACKER\", \"rule.name\": \"ET SCAN Nmap OS Detection\",  
\"suricata.eve.alert.category\": \"attempted-recon\" }" > /dev/null # Phase 2 – SSH  
Brute Force for i in {1..15}; do sshpass -p "wrongpass" ssh -o StrictHostKeyChecking=no  
\ -o ConnectTimeout=2 testuser@$TARGET 2>/dev/null sleep 1 done # Phase 3 – SQLi curl -s  
-X POST "$ES/$IDX/_doc" -u "$AUTH" \ -H "Content-Type: application/json" -d "{  
\"@timestamp\": \"$(date -u +%Y-%m-%dT%H:%M:%SZ)\", \"event.module\": \"suricata\",  
\"source.ip\": \"$ATTACKER\", \"rule.name\": \"ET SQL SQL Injection Attack\",  
\"suricata.eve.alert.category\": \"web-application-attack\" }" > /dev/null echo  
"Simulation terminée" SCRIPT chmod +x /tmp/simulate_attack.sh bash  
/tmp/simulate_attack.sh
```

10 — PROBLÈME : SURICATA/PFSENSE NE DÉTECTE PAS

Architecture et limitation

Suricata sur pfSense ne détecte que le trafic qui **traverse** pfSense. Si Kali et VM-WEB sont sur le même LAN (192.168.1.0/24), le trafic passe directement entre eux sans traverser pfSense.

Scénario	Kali → VM-WEB	pfSense voit ?	Suricata alerte ?
Kali sur LAN (192.168.1.x)	Direct L2	■ Non	■ Non
Kali sur WAN (10.12.x.x)	Via pfSense	■ Oui	■ Oui
Kali avec route forcée	Via pfSense GW	■■ Partiel	■■ Partiel

■ Solution recommandée pour le lab

Utiliser Wazuh Agent sur VM-WEB pour détecter les attaques SSH/web directement sur la machine cible — indépendant de pfSense. Cela couvre : SSH brute force (rule 5712), scans web (Apache logs), modifications de fichiers (FIM).

pfSense → envoyer syslog vers VM-SIEM

```
Dans pfSense : Status → System Logs → Settings → Enable Remote Logging : cocher →  
Remote log servers : 192.168.1.200:514 → Remote Syslog Contents : Everything → Save #  
Vérifier réception sur VM-SIEM sudo tcpdump -i any udp port 514 -c 5
```

11 — CRÉER DATA VIEW DANS KIBANA

Sans Data View, les alertes Wazuh n'apparaissent pas dans Discover.

Étape	Action
1	Kibana → Stack Management (menu bas gauche)
2	Data Views → Create data view
3	Name : Wazuh Alerts
4	Index pattern : wazuh-alerts-*
5	Timestamp field : @timestamp
6	Save to Kibana
7	Discover → Sélectionner "Wazuh Alerts"
8	Ajouter colonnes : data.srcip, rule.description, agent.name

12 — ERREURS FRÉQUENTES ET SOLUTIONS

■ ■ 400 Bad Request / undefined: undefined

Fleet essaie de contacter Elastic Cloud. Erreur cosmétique bénigne en lab offline.

Solution : accéder directement via `http://localhost:5601/app/security`

■ Detection engine permissions required / 403 Forbidden

xpack.security.enabled doit être true dans `elasticsearch.yml`.

Solution : activer la sécurité ES et créer les mots de passe.

■ ECONNREFUSED 127.0.0.1:9200

Elasticsearch n'est pas démarré ou a crashé.

Solution : `systemctl start elasticsearch && journalctl -u elasticsearch -n 20`

■ ■ Wazuh rule 5712 ne se déclenche pas

`/var/log/auth.log` n'est pas surveillé par défaut dans Wazuh.

Solution : ajouter la localfile `auth.log` dans `ossec.conf` sur VM-WEB.

■ ■ source.ip vide dans les alertes

Wazuh utilise `data.srcip` au lieu de `source.ip` (format ECS).

Solution : dans Kibana Alerts → Fields → ajouter `data.srcip`.

■ Security Alerts vide malgré des données dans ES

Les règles de détection ne sont pas activées.

Solution : Security → Rules → Create rule OU activer les règles prédéfinies.

■ ■ elasticsearch-setup-passwords : Connection refused

ES utilise `elasticsearch-reset-password` dans les versions récentes.

Solution : `elasticsearch-reset-password -u elastic`

■ File realm configuration file missing

Les fichiers `/etc/elasticsearch/users` et `users_roles` sont absents.

Solution : `sudo touch /etc/elasticsearch/users users_roles && chown root:elasticsearch`

13 — COMMANDES DE RÉFÉRENCE RAPIDE

Commande	Description
<code>systemctl restart elasticsearch</code>	Redémarrer Elasticsearch
<code>systemctl restart kibana</code>	Redémarrer Kibana
<code>systemctl restart wazuh-manager</code>	Redémarrer Wazuh Manager
<code>systemctl restart wazuh-agent</code>	Redémarrer Wazuh Agent
<code>systemctl restart filebeat</code>	Redémarrer Filebeat
<code>/var/ossec/bin/agent_control -l</code>	Lister agents Wazuh
<code>curl -s http://localhost:9200/ -u elastic:PASS</code>	Tester ES
<code>curl -s http://localhost:5601/api/status</code>	Tester Kibana
<code>curl -s "http://localhost:9200/_cat/indices?v" -u elastic:PASS</code>	Lister les index ES
<code>journalctl -u elasticsearch -n 50 --no-pager</code>	Logs ES
<code>journalctl -u kibana -n 50 --no-pager</code>	Logs Kibana
<code>tail -f /var/ossec/logs/alerts/alerts.json</code>	Alertes Wazuh temps réel
<code>tail -f /var/log/auth.log</code>	Logs SSH temps réel
<code>ss -ltnp grep -E "9200 5601 1514"</code>	Ports ouverts
<code>ip a grep "inet "</code>	IPs des interfaces
<code>sudo netplan apply</code>	Appliquer config réseau